
Universidad del Valle de Guatemala

Computación Paralela y Distribuida

Laboratorio 3

Comunicación bloqueante y no bloqueante con MPI

Estudiante:	Daniel Barillas
Carné:	22193
Sección:	40
Docente (enunciado):	Marlon Fuentes
Entrega vigente:	Sábado 3 de octubre de 2026, 23:59

Borrador técnico. Las evidencias, gráficas y conclusiones experimentales se completarán después de la ejecución del estudiante en WSL.

Resumen

Este informe documenta el diseño de cuatro programas de comunicación punto a punto con MPI. Los dos primeros emplean llamadas bloqueantes para resolver un intercambio Ping-Pong y un anillo con un único token. Los dos restantes estudian comunicación no bloqueante mediante una recepción anticipada con trabajo aritmético y un productor-consumidor que transfiere un arreglo por chunks. Se define un protocolo reproducible de medición con `MPI_Wtime`, validación de datos y repeticiones por configuración. Las observaciones cuantitativas se incorporarán cuando existan resultados obtenidos por el estudiante; este borrador no presupone ninguna tendencia.

Contenido

1	Introducción y alcance	3
2	Fundamentos de comunicación MPI	3
2.1	Llamadas bloqueantes	3
2.2	Llamadas no bloqueantes	3
2.3	Medición temporal	4
3	Entorno y método experimental	4
3.1	Presentación de consola y trazabilidad	4
4	Parte 1: comunicación con bloqueo	5
4.1	Ping-Pong	5
4.2	Token Ring	6
5	Parte 2: comunicación sin bloqueo	6
5.1	Recepción anticipada	6
5.2	Productor-consumidor: pipeline por chunks	7
6	Discusión metodológica	8
7	Conclusiones	8
8	Referencias	8
9	Apéndice: comandos de referencia	9
9.1	Archivos que se incorporarán al cierre	9

1 Introducción y alcance

MPI (Message Passing Interface) permite que procesos independientes intercambien datos mediante mensajes. Cada proceso conoce su identificador mediante `MPI_Comm_rank` y el tamaño de su comunicador mediante `MPI_Comm_size`. En los cuatro programas se utiliza `MPI_COMM_WORLD`, que agrupa a los procesos iniciados por `mpirun`. La coordinación correcta depende de emparejar origen, destino, etiqueta y comunicador en cada transferencia.

El enunciado divide el trabajo en dos partes: comunicación con bloqueo y comunicación sin bloqueo. Cada parte contiene dos ejercicios. La tabla de evaluación del PDF muestra cuatro partes de 25 %, aunque solo las primeras dos están nombradas en el texto; por ello este informe presenta cuatro secciones de ejercicios sin asignarles oficialmente un porcentaje.

Ejercicio	Comunicación	Evidencia requerida
Ping-Pong	<code>MPI_Send</code> y <code>MPI_Recv</code>	Tiempo frente a tamaño de mensaje
Token Ring	<code>MPI_Sendrecv</code> y variante separada	Recorrido correcto y explicación de diferencias
Recepción anticipada	<code>MPI_Irecv</code> , <code>MPI_Isend</code> , <code>MPI_Test</code>	Trabajo efectuado y tiempo frente a tamaño
Pipeline por chunks	<code>MPI_Isend</code> , recepción por chunk y STOP	Tiempo total frente a tamaño de chunk

2 Fundamentos de comunicación MPI

2.1 Llamadas bloqueantes

En una recepción bloqueante, la llamada retorna cuando el buffer contiene el mensaje recibido. En un envío bloqueante, el retorno indica que el buffer del emisor puede reutilizarse; no implica por sí solo que el receptor haya terminado de procesar los datos. La pareja de llamadas exige un orden compatible entre procesos. Si todos los participantes esperan una operación que nadie puede completar, se produce un bloqueo circular.

`MPI_Sendrecv` combina en una sola llamada un envío y una recepción. La operación es bloqueante, pero el subsistema MPI coordina ambos lados y facilita desplazamientos en topologías como un anillo. Los buffers de envío y recepción deben ser distintos. La alternativa con `MPI_Send` y `MPI_Recv` separados requiere planear explícitamente el orden.

2.2 Llamadas no bloqueantes

`MPI_Irecv` y `MPI_Isend` crean una petición y retornan antes de que necesariamente termine la transferencia. `MPI_Test` consulta una petición sin esperar hasta su finalización. Esto permite alternar comunicación y cálculo dentro del mismo proceso. Una petición de envío debe completarse antes de modificar o liberar su buffer.

2.3 Medición temporal

`MPI_Wtime` entrega segundos transcurridos según el reloj local del proceso. Cada duración se calcula restando dos lecturas tomadas en el *mismo* rank; no se restan marcas temporales de ranks distintos. Se registra una ronda de calentamiento en Ping-Pong y se realizan tres repeticiones por configuración. La mediana atenúa corridas aisladas con interferencia del sistema, pero la dispersión seguirá siendo relevante.

3 Entorno y método experimental

Las ejecuciones están previstas en Ubuntu bajo WSL con Open MPI. El estudiante compilará los cuatro programas con `mpicc` y el `Makefile` del repositorio. Se registrarán las versiones de `mpicc`, `mpirun` y el equipo utilizado. La ejecución se realizará localmente en una sola máquina; `-oversubscribe` se usará solo si el anillo de cinco procesos excede los slots disponibles.

Pendiente de evidencia: Versión de Open MPI, distribución WSL y características del equipo. Se incorporará únicamente con las mediciones reales del estudiante.

Para cada experimento se mantiene fijo el trabajo total pertinente y se varía una sola magnitud principal. Los programas imprimen una línea CSV al final, además de un resumen legible. Las capturas documentan la ejecución y los CSV conservan los números exactos para las gráficas. Las figuras se elaborarán a partir de la mediana de tres repeticiones; si hay variabilidad apreciable, se mostrarán los valores individuales.

3.1 Presentación de consola y trazabilidad

Los cuatro programas comparten una presentación textual en terminal: encabezados azules, rutas de mensajes cian, comprobaciones verdes y notas amarillas. Los diagramas explican el recorrido de los datos, pero no representan una escala física de tiempo. Las barras de veinte celdas indican finalización o visitas verificadas; la medición numérica siempre aparece por separado. La variable `NO_COLOR` elimina los códigos ANSI cuando se preparan registros de texto. La última línea se conserva como CSV puro para evitar transcribir tiempos desde las capturas.

La implementación separa la lógica MPI de la presentación mediante `common.h` y `visual.h`. Cada programa documenta sus argumentos, roles de los ranks, etiquetas, buffers, orden de llamadas, verificación y límites de la región cronometrada. Los dibujos se imprimen *después* de calcular el tiempo con `MPI_Wtime`; así no alteran las comparaciones solicitadas.

Experimento	Variable independiente	Control
Ping-Pong	1, 64, 1024, 16384, 65536 enteros	1000 rondas, 2 ranks
Token Ring	Dos variantes de comunicación	5 ranks, 1000 vueltas
Recepción anticipada	Los mismos tamaños de mensaje	1000 operaciones por consulta, 2 ranks
Pipeline	Chunks de 256 a 65536 enteros	1 048 576 enteros totales, 2 ranks

4 Parte 1: comunicación con bloqueo

4.1 Ping-Pong

Pendiente de evidencia: Captura 01_ping_pong_entero.png. Intercambio básico de un entero durante N rondas.

Pendiente de evidencia: Captura 02_ping_pong_tamanos.png. Comparación de al menos dos tamaños de mensaje.

El rank 0 genera un buffer de enteros y lo envía a rank 1. Este lo recibe y lo devuelve sin modificarlo. Una ronda consta de un envío y una devolución; el usuario fija N mediante el primer argumento. Para cumplir simultáneamente el caso básico y el estudio de tamaños, el segundo argumento controla el número de enteros: con valor 1 se envía un entero, y con valores mayores se envía un arreglo de enteros. El último mensaje devuelto se compara con el patrón original antes de informar que la prueba fue correcta.

La implementación separa las dos perspectivas en `round_as_origin` y `round_as_echo`. La salida dibuja PING y PONG con etiquetas diferentes, muestra N rondas completadas y presenta el tiempo solo después de verificar el mensaje. Esto permite relacionar la captura con las llamadas MPI del código.

La duración total medida en rank 0 se divide entre N para obtener la latencia media de ida y vuelta:

$$T_{\text{RTT}} = \frac{t_{\text{fin}} - t_{\text{inicio}}}{N}, \quad T_{\text{ida,estimada}} = \frac{T_{\text{RTT}}}{2}.$$

La segunda expresión es una estimación bajo simetría aproximada, no una medición independiente de una dirección. La gráfica solicitada usará bytes por mensaje en el eje horizontal y T_{RTT} en el vertical.

Pendiente de evidencia: Tabla, gráfica tamaño de mensaje frente a tiempo y discusión de tendencia del Ping-Pong. Se incorporará únicamente con las mediciones reales del estudiante.

4.2 Token Ring

Pendiente de evidencia: Captura 03_token_ring_send_recv.png. Anillo con envíos y recepciones separados.

Pendiente de evidencia: Captura 04_token_ring_sendrecv.png. Anillo con la instrucción MPI_Sendrecv.

Se usan al menos cinco ranks. Para un rank i de un comunicador de tamaño P , el vecino anterior y el siguiente son

$$\text{src}(i) = (i - 1 + P) \bmod P, \quad \text{dst}(i) = (i + 1) \bmod P.$$

Rank 0 comienza con un único token válido. Cada recepción incrementa el valor, lo cual permite verificar que el token visitó todos los procesos. Tras V vueltas, el token devuelto a rank 0 debe valer PV , y cada rank debe registrar exactamente V visitas.

En la primera variante, rank 0 envía para iniciar cada vuelta y después recibe el token de su vecino anterior. Los otros ranks reciben primero y luego envían. Este orden evita que todos queden esperando dentro de un envío bloqueante. En la segunda variante, todos los ranks ejecutan MPI_Sendrecv en cada paso: el rank que no posee el token envía un centinela -1 , mientras el poseedor envía el valor válido. Una vuelta consta de P pasos; sigue existiendo un único token válido.

La diferencia central es que MPI_Sendrecv combina las dos operaciones bloqueantes en una llamada y coordina su progreso. Con MPI_Send y MPI_Recv separados, el programador debe establecer el orden seguro de cada rank. El centinela hace explícito el estado vacío del anillo y permite que todos llamen a MPI_Sendrecv en cada paso, aunque solo un mensaje contenga el token verdadero.

El resumen visual dibuja la ruta $0 \rightarrow 1 \rightarrow \dots \rightarrow P - 1 \rightarrow 0$ y una barra de visitas por rank. Las barras confirman el contador esperado, mientras el valor numérico final detecta un salto omitido o duplicado.

Pendiente de evidencia: Comparación de ambas variantes y verificación de visitas por rank. Se incorporará únicamente con las mediciones reales del estudiante.

5 Parte 2: comunicación sin bloqueo

5.1 Recepción anticipada

Pendiente de evidencia: Captura 05_recepcion_entero.png. Recepción básica de un entero y trabajo aritmético.

Pendiente de evidencia: Captura 06_recepcion_tamanos.png. Ejemplo con un mensaje de mayor tamaño.

Rank 1 inicia `MPI_Irecv` antes de hacer aritmética. A continuación ejecuta un lote fijo de operaciones y llama a `MPI_Test`; repite esta secuencia hasta que la petición se completa. Rank 0 prepara el buffer, inicia `MPI_Isend` y espera antes de liberarlo. El caso de un elemento cumple el intercambio de un entero y los arreglos permiten estudiar mensajes mayores sin cambiar el protocolo.

El receptor registra el primer entero, la suma validada, el número total de operaciones aritméticas y el tiempo transcurrido desde antes de publicar la recepción hasta que `MPI_Test` informa su finalización. También se registra por separado el tiempo local de envío del rank 0. El tiempo del receptor incluye cómputo y sondeo; por ello no debe interpretarse como latencia pura de red. La cantidad de trabajo realizado antes de recibir puede cambiar entre corridas según la planificación de procesos.

La línea de tiempo de la terminal distingue el envío de rank 0 de la secuencia `Irecv` → cálculo → `MPI_Test` de rank 1. `do_arithmetic` mantiene visible el estado del cálculo y `verify_and_sum` recorre todos los enteros después de completar la petición.

Pendiente de evidencia: Gráfica de tamaño de mensaje frente a tiempo y análisis del trabajo solapado. Se incorporará únicamente con las mediciones reales del estudiante.

5.2 Productor-consumidor: pipeline por chunks

Pendiente de evidencia: Captura 07_pipeline_chunks.png. Ejecución con chunks, mensaje STOP y suma correcta.

Pendiente de evidencia: Captura 08_pipeline_tamanos.png. Ejecución con un tamaño de chunk distinto.

Rank 0 crea un arreglo de 1 048 576 enteros. El tamaño de chunk debe dividir al total exactamente; por ello todos los bloques tienen la misma cantidad de elementos. El productor conserva hasta dos envíos `MPI_Isend` pendientes. Antes de reutilizar una petición, espera su finalización. Rank 1 detecta el siguiente mensaje, recibe el chunk y procesa sus enteros de inmediato. Al terminar todos los envíos, rank 0 envía mediante `MPI_Isend` un mensaje de control STOP. El receptor lo reconoce por su etiqueta y sale del bucle.

Cada entero se compara con el patrón esperado y se acumula una suma. Al final, el consumidor comunica al productor el número de chunks y su suma. Rank 0 compara ambos contra los valores previstos. El tiempo total se toma en rank 0 y termina después de la confirmación del consumidor y una barrera final; incluye transferencia y procesamiento del lote completo.

La figura textual muestra varios chunks en el trayecto productor-consumidor y un segundo mensaje de control STOP. La función `produce` administra las dos peticiones pendientes; `consume` inspecciona etiqueta y tamaño de cada mensaje, procesa el bloque y sale del bucle únicamente tras recibir STOP. La figura se genera después de la medición.

El número de chunks es $C = M/B$, donde M es la cantidad total de enteros y B el tamaño de chunk en enteros. Chunks pequeños implican más mensajes y posiblemente mayor costo de coordinación; chunks grandes reducen el número de mensajes, pero pueden limitar el solapamiento. La tendencia concreta debe decidirse con los datos observados.

Pendiente de evidencia: Tabla y gráfica de tamaño de chunk frente a tiempo total, más discusión. Se incorporará únicamente con las mediciones reales del estudiante.

6 Discusión metodológica

Los experimentos sobre una sola máquina y WSL pueden reflejar costos de memoria compartida, planificación del sistema operativo y configuración de Open MPI, además de los costos del protocolo. El uso de `-oversubscribe` en Token Ring también puede introducir espera por CPU; por ello los tiempos entre ejercicios no se compararán como si midieran un mismo recurso. Cada gráfica responderá a la pregunta concreta del ejercicio y señalará su escala y unidad.

Las salidas verificadas son una condición previa para interpretar los tiempos. Si una corrida no finaliza correctamente, se conserva el registro del error pero se excluye de la estadística hasta resolver su causa. Las tres repeticiones por configuración son un mínimo práctico; si hay alta dispersión, convendrá repetir más veces el caso afectado.

7 Conclusiones

El diseño cubre los cuatro ejercicios del enunciado con parámetros controlables, comprobaciones de corrección y un protocolo explícito de medición. La comunicación bloqueante exige coordinar el orden de envíos y recepciones; `MPI_Sendrecv` simplifica el desplazamiento circular. Las operaciones no bloqueantes permiten iniciar comunicación y continuar el cálculo o la producción de datos, siempre que se controle la vida de los buffers y la finalización de las peticiones.

Pendiente de evidencia: Conclusiones cuantitativas sustentadas en las gráficas y capturas del estudiante. Se incorporará únicamente con las mediciones reales del estudiante.

8 Referencias

1. Universidad del Valle de Guatemala. *Laboratorio #3*, Computación Paralela y Distribuida, semestre 2 de 2025. Enunciado PDF proporcionado para esta realización.

2. Open MPI. *MPI_Sendrecv*. Manual oficial. https://docs.open-mpi.org/en/main/man-openmpi/man3/MPI_Sendrecv.3.html.
3. Open MPI. *MPI_Test*. Manual oficial. https://docs.open-mpi.org/en/main/man-openmpi/man3/MPI_Test.3.html.
4. Open MPI. *MPI_Wtime*. Manual oficial. https://docs.open-mpi.org/en/main/man-openmpi/man3/MPI_Wtime.3.html.

9 Apéndice: comandos de referencia

```
make
mpirun -np 2 ./build/ping_pong 10 1
mpirun -oversubscribe -np 5 ./build/token_ring sendrecv 1
mpirun -np 2 ./build/recepcion_anticipada 1 1000
mpirun -np 2 ./build/pipeline_chunks 1048576 4096
bash scripts/mediciones.sh
```

La última instrucción ejecuta el barrido de mediciones del estudiante, genera cuatro CSV y conserva las salidas completas de cada corrida.

9.1 Archivos que se incorporarán al cierre

Archivo	Uso en el informe final
ping_pong.csv	Gráfica de bytes frente a ida y vuelta media
token_ring.csv	Comparación descriptiva de las dos variantes
recepcion_anticipada.csv	Tiempo y trabajo efectuado según bytes
pipeline_chunks.csv	Gráfica de bytes por chunk frente a tiempo total
Capturas PNG	Evidencia visible antes del análisis de cada apartado

Cada CSV incluye tres repeticiones por configuración. Los registros originales de terminal se conservan por separado, de modo que cualquier valor de una figura pueda rastrearse hasta el comando y la ejecución que lo produjo. Las capturas y datos se incorporarán solo después de que el estudiante realice las pruebas en WSL.